

DOCKER & DEVOPS

Scenario-Based Practice Questions — Volume 2

8 New Scenarios · Complete Solutions · LPU DevOps Lab

Docker Hub & Registry

Container Linking

Microservices

YAML Deep Dive

Swarm Stacks

Jenkins Nodes

Bind Mounts

Multi-stage Builds

Volume 2 continues with eight more scenario-based questions covering Docker Hub and image registries, container linking, microservice communication, YAML deep dives, Swarm stack deployments, Jenkins agent nodes, bind mounts for live development, and multi-stage Dockerfile builds for production-grade images.

Push a Custom Image to Docker Hub

SCENARIO

Your team has built a custom nginx image with a branded error page. You need to tag it properly and push it to your Docker Hub account so that any team member globally can pull it with a single command. The image must be versioned as 1.0 and also tagged as 'latest'.

TASKS

1. Log in to Docker Hub from the command line.
2. Tag the local image branded-nginx:build as yourusername/branded-nginx:1.0 and :latest.
3. Push both tags to Docker Hub.
4. Pull the image on a different machine and verify it runs correctly.

SOLUTION

Replace 'yourusername' with your actual Docker Hub username throughout.

```
# Step 1 – log in to Docker Hub
docker login
# Enter your Docker Hub username and password (or access token)

# Step 2 – tag the local image with two tags
docker tag branded-nginx:build yourusername/branded-nginx:1.0
docker tag branded-nginx:build yourusername/branded-nginx:latest

# Step 3 – push both tags
docker push yourusername/branded-nginx:1.0
docker push yourusername/branded-nginx:latest

# Step 4 – verify from another machine (or after docker rmi locally)
docker rmi yourusername/branded-nginx:1.0
docker pull yourusername/branded-nginx:1.0
docker run -d -p 80:80 yourusername/branded-nginx:1.0
curl http://localhost
```

KEY CONCEPT

Docker Hub is a public registry. An image name on Hub follows the pattern username/repository:tag. Pushing :latest alongside a version tag is best practice — :latest lets teammates pull the newest build without knowing the version number, while the numbered tag lets you roll back to any specific release.

Q1 0

Container Linking

Legacy Links & Bridge Networks

Connect a Flask App to Redis Without Compose

SCENARIO

A developer wrote a Flask hit-counter app that stores counts in Redis. You must run both containers on the same machine and ensure Flask can reach Redis using the hostname 'redis-cache' — without using Docker Compose. Both containers must be on a custom bridge network.

TASKS

1. Create a custom bridge network called app-net.
2. Run a Redis container named redis-cache on app-net.
3. Run the Flask app container on app-net with the correct environment variable pointing to Redis.
4. Test that Flask can reach Redis by curling the hit-counter endpoint.

SOLUTION

```
# Step 1 – create a custom bridge network
docker network create app-net
```

```
# Step 2 – run Redis on the network, named 'redis-cache'
docker run -d \
  --name redis-cache \
  --network app-net \
  redis:7
```

```
# Step 3 – run Flask app on the same network
# The app reads REDIS_HOST env var to connect
docker run -d \
  --name flask-counter \
  --network app-net \
  -e REDIS_HOST=redis-cache \
  -e REDIS_PORT=6379 \
  -p 5000:5000 \
  myapp/flask-counter:latest
```

```
# Step 4 – test the endpoint
curl http://localhost:5000/count
# Expected: {"hits": 1}

# Verify DNS resolution inside the Flask container
docker exec flask-counter ping -c 2 redis-cache
```

KEY CONCEPT

On a user-defined bridge network, Docker provides automatic DNS resolution — every container can reach others by their `--name`. This replaces the older `--link` flag which is now deprecated. Using a custom network also isolates these containers from unrelated containers on the default bridge.

Q1 1

Docker Networks

Network Inspection & Troubleshooting

Two Containers Cannot Talk to Each Other

SCENARIO

A colleague deployed an API container and a database container but they cannot communicate. After checking, you discover they are on different networks. You must diagnose the problem, connect both to a shared network, and document the correct network inspection workflow.

TASKS

1. List all Docker networks and inspect which network each container is on.
2. Create a shared network called shared-net.
3. Connect both containers to shared-net without restarting them.
4. Verify connectivity using docker exec ping from one container to the other.

SOLUTION

```
# Step 1 – list networks and see which containers are on each
docker network ls
docker network inspect bridge

# Or inspect a specific container's network
docker inspect api-container | grep -A 20 '"Networks'"
```

```
# Step 2 – create the shared network
docker network create shared-net
```

```
# Step 3 – connect BOTH containers to shared-net (no restart needed)
docker network connect shared-net api-container
docker network connect shared-net db-container
```

```
# Step 4 – test connectivity
docker exec api-container ping -c 3 db-container
# Should resolve by name and get responses
# Full network inspection
docker network inspect shared-net
```

KEY CONCEPT

docker network connect attaches a running container to an additional network instantly — no restart required. A container can be on multiple networks simultaneously. docker network inspect shows every container currently connected and their assigned IPs, making it the first tool to reach for when diagnosing connectivity failures.

Q1

2

Microservices

Multi-container Microservice Architecture

Deploy a 3-Service E-Commerce Backend

SCENARIO

Your startup is splitting a monolith into microservices. The first milestone is three services: user-service (Node.js on port 3001), product-service (Python Flask on port 3002), and order-service (Java on port 3003) — each with its own database. All services must communicate over a private network but only order-service is exposed to the public on port 8080.

TASKS

1. Write a docker-compose.yml with all three services and their dedicated databases.
2. Create an internal network (backend-net) and ensure only order-service is published externally.
3. Add health checks to all three databases.
4. Confirm inter-service communication by exec-ing into order-service and curling user-service.

SOLUTION

Save as docker-compose.yml and run: docker compose up -d

```
version: "3.9"

networks:
  backend-net:
    driver: bridge

volumes:
  users_db_data:
  products_db_data:
  orders_db_data:

services:
  user-service:
    image: node:18
    container_name: user-svc
    working_dir: /app
    volumes: ["/user-service:/app"]
    command: sh -c "npm install && npm start"
    networks: [backend-net]
    depends_on:
      users-db:
        condition: service_healthy
  users-db:
    image: postgres:15
    environment:
      POSTGRES_DB: usersdb
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes: ["users_db_data:/var/lib/postgresql/data"]
    networks: [backend-net]
    healthcheck:
```

```

test: ["CMD-SHELL","pg_isready -U user"]
interval: 5s
retries: 5
product-service:
image: python:3.11-slim
container_name: product-svc
working_dir: /app
volumes: ["/product-service:/app"]
command: sh -c "pip install -r requirements.txt && python app.py"
networks: [backend-net]
depends_on:
products-db:
condition: service_healthy

products-db:
image: mysql:8
environment:
MYSQL_DATABASE: productsdb
MYSQL_ROOT_PASSWORD: rootpass
volumes: ["products_db_data:/var/lib/mysql"]
networks: [backend-net]
healthcheck:
test: ["CMD","mysqladmin","ping","-h","localhost"]
interval: 10s
retries: 5

order-service:
image: eclipse-temurin:17
container_name: order-svc
working_dir: /app
volumes: ["/order-service:/app"]
command: java -jar app.jar
networks: [backend-net]
ports:
- "8080:8080" # Only this service is exposed!
environment:
USER_SERVICE_URL: http://user-svc:3001
PRODUCT_SERVICE_URL: http://product-svc:3002
depends_on:
orders-db:
condition: service_healthy

orders-db:
image: mongo:7
volumes: ["orders_db_data:/data/db"]
networks: [backend-net]
healthcheck:
test: ["CMD","mongosh","--eval","db.adminCommand('ping')"]
interval: 10s
retries: 5

# Verify inter-service DNS resolution

```

```
docker exec order-svc curl -s http://user-svc:3001/health
docker exec order-svc curl -s http://product-svc:3002/health
# Check all services are healthy
docker compose ps
```

KEY CONCEPT

By not publishing ports for user-service and product-service, they are reachable only by other containers on backend-net — not from the internet. order-service acts as the API gateway, publishing port 8080 publicly while calling the other services internally by their Compose service names.

Q1 3

Stop Hardcoding Passwords in docker-compose.yml

SCENARIO

During a security audit, the auditor flags that your docker-compose.yml has plain-text database passwords committed to Git. You must refactor it to use a .env file for non-sensitive config and Docker secrets for sensitive credentials, then prove the passwords no longer appear in the Compose file.

TASKS

1. Create a .env file with non-sensitive variables (ports, DB names) and add it to .gitignore.
2. Rewrite docker-compose.yml to use variable substitution with \${VAR_NAME} syntax.
3. Use Docker secrets for the database password in a Swarm stack (secrets: block).
4. Verify the .env variables are picked up correctly with docker compose config.

SOLUTION

Never commit .env or secrets files to Git. Add them to .gitignore immediately.

```
# .env (never commit this file)
DB_NAME=mydb
DB_USER=appuser
DB_PORT=5432
APP_PORT=8080
POSTGRES_IMAGE=postgres:15

# docker-compose.yml (safe to commit – no secrets)
version: "3.9"
services:
  app:
    image: myapp:latest
    ports:
      - "${APP_PORT}:8080"
    environment:
      DB_HOST: db
      DB_NAME: ${DB_NAME}
      DB_USER: ${DB_USER}
      DB_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    depends_on:
      - db
  db:
    image: ${POSTGRES_IMAGE}
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
```



```

ports:
- "${DB_PORT}:5432"

secrets:
db_password:
file: ./secrets/db_password.txt
# secrets/db_password.txt (never commit)
# SuperSecretPass123!

# Create the secrets folder and password file
mkdir secrets
echo 'SuperSecretPass123!' > secrets/db_password.txt
echo 'secrets/' >> .gitignore
echo '.env' >> .gitignore

# Verify variable substitution
docker compose config
# This prints the fully-resolved compose file – confirm no raw passwords

```

KEY CONCEPT

Variable substitution (`${VAR}`) reads from `.env` automatically when Compose starts. Docker secrets mount a file at `/run/secrets/secret_name` inside the container — the app reads the password from that file, so it never appears as an env var (which could be leaked via `docker inspect`). This is the production-safe pattern.

Q1

Orchestration

Swarm Stack Deployment

4

Deploy the Full Stack as a Swarm Stack

SCENARIO

Your application is ready for production. Instead of `docker compose up`, the ops team wants it deployed as a Docker Swarm stack so it gets load balancing and self-healing for free. You must convert the Compose file into a stack, deploy it, update the app image with zero downtime, and roll back if something goes wrong.

TASKS

1. Deploy the application as a Swarm stack named 'prod' from the `docker-compose.yml`.
2. Scale the app service to 3 replicas with a rolling update policy.
3. Simulate a bad deploy by pushing a broken image, then roll back to the previous version.
4. List all running stacks and services to confirm the healthy state.

SOLUTION

Swarm must be initialised first: `docker swarm init`

```

# Deploy the stack (Swarm reads the Compose file)
docker stack deploy -c docker-compose.yml prod

# Check stack services
docker stack services prod

# Check individual tasks (containers)

```

```
docker stack ps prod
```

```
# Scale app service to 3 replicas
docker service scale prod_app=3

# Or add update_config in compose file for rolling updates:
# deploy:
#   replicas: 3
#   update_config:
#     parallelism: 1
#     delay: 10s
#     failure_action: rollback
#   rollback_config:
#     parallelism: 1
```

```
# Update to a new image (rolling – zero downtime)
docker service update \
  --image myapp:2.0 \
  --update-parallelism 1 \
  --update-delay 10s \
  prod_app
```

```
# If the new image is broken, roll back to previous
docker service rollback prod_app

# Verify healthy state
docker service ps prod_app
docker stack ls
```

KEY CONCEPT

docker stack deploy re-uses the Compose file format but runs under Swarm's orchestrator. Rolling updates replace one replica at a time (parallelism: 1), so there is always at least one healthy replica serving traffic during the update. docker service rollback instantly reverts to the previous image and configuration without needing to know what the old image tag was.

Q1 5

Storage

Bind Mounts for Live Development

Code Changes Should Reflect Instantly Without Rebuilding

SCENARIO

Your frontend developers are frustrated — every CSS change requires docker build which takes 2 minutes. You must set up a bind mount so the container reads source files directly from the host. Changes on disk must be visible in the browser immediately, with no rebuild.

TASKS

1. Run the Node.js dev server with the project folder bind-mounted into /app.
2. Explain the difference between a bind mount and a named volume.
3. Ensure node_modules inside the container is NOT overwritten by the host mount.
4. Test by editing a source file on the host and confirming the change is live.

SOLUTION

Bind mounts use absolute paths. Use \$(pwd) on Linux/Mac, %cd% on Windows CMD, or \${PWD} on PowerShell.

```
# Run with bind mount – host ./src is live-synced to /app in the container
docker run -d \
  --name dev-frontend \
  -p 3000:3000 \
  -v $(pwd)/src:/app/src \
  -v $(pwd)/public:/app/public \
  node:18 \
  sh -c "npm install && npm start"
```

```
# IMPORTANT: prevent host's (empty) node_modules overwriting container's
# Add an anonymous volume for node_modules – it takes precedence
docker run -d \
  --name dev-frontend \
  -p 3000:3000 \
  -v $(pwd):/app \
  -v /app/node_modules \
  node:18 \
  sh -c "npm install && npm start"

# The naked -v /app/node_modules creates an anonymous volume
# that shadows the bind mount at that specific sub-path
```

```
# Difference – named volume vs bind mount:
# Named volume: -v mydata:/app/data
# - Managed by Docker, stored in /var/lib/docker/volumes/
# - Best for: databases, persistent state
# - Content NOT visible/editable from host easily
#
# Bind mount: -v /host/path:/container/path
# - Points directly to a host directory
# - Best for: source code, config files, live dev
```

```
# Refresh http://localhost:3000 – background is now red, no rebuild needed
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

```
# Build the multi-stage image
```

```
docker build -t myapp:slim .
```

```
# Compare sizes – should drop from ~1.2GB to ~200MB
```

```
docker images myapp
```

```
# REPOSITORY TAG IMAGE ID SIZE
```

```
# myapp slim abc123def456 198MB <- final image
```

```
# (build stage is discarded automatically)
```

```
# Push the slim image to Docker Hub
```

```
docker tag myapp:slim yourusername/myapp:slim
```

```
docker push yourusername/myapp:slim
```

KEY CONCEPT

Multi-stage builds use multiple FROM instructions in one Dockerfile. Each stage starts fresh. COPY --from=build copies only the artifact you need from an earlier stage — Maven, the .m2 cache, and all source files stay in the build stage which Docker discards after the build completes. The final Alpine-based JRE image is typically 80-85% smaller than a full JDK image and has a much smaller CVE attack surface.